Übungsblatt 1: Speicherphysik & RAM-Typen

Mikrocomputertechnik 3 · Zellzustände, Ladung und Adressen auf Papier

Musterlösung

Ziel und Arbeitsweise

Sechs Pflichtaufgaben sind vollständig auf Papier lösbar und üben Begründungen, Skizzen und Rechenwege wie in einer Klausur. Alle Zahlen und Adressbereiche sind ausdrücklich Lehrmodelle. Geben Sie Annahmen, Zwischenwerte und Einheiten an.

Bearbeitungszeit: etwa 75–90 Minuten selbstständig. **Orientierungspunkte:** 32, keine neue Prüfungsregel. Im Unterricht: Aufgaben 1–2 (10 min), Aufgabe 3 (15 min), Aufgabe 4 (15 min), Aufgaben 5–6 (15 min). Die Vorlesungsphasen enthalten Lösungsbesprechungen.

SRAM = Static Random-Access Memory, statischer wahlfreier Speicher. DRAM = Dynamic Random-Access Memory, dynamischer wahlfreier Speicher. 6T = sechs Zelltransistoren, 1T1C = ein Transistor und ein Kondensator. WL = Word Line, Wortleitung. BL / BLB = komplementäre Bit Lines, Bitleitungen. KiB = 2^{10} Byte, MiB = 2^{20} Byte, ein Byte enthält acht Bit. Die professionelle Speicherentscheidung unterscheidet Zelle, Zugriff und Software.

Aufgabe 1: Kapazität und gemeinsame Wege (5 Punkte)

1. Ein Speicher enthält 1 MiB Nutzdaten. Berechnen Sie Byte- und Bitzahl. Wie unterscheiden sich MiB und MB? (1 P)
2. Zählen Sie dafür die Zelltransistoren bei klassischem 6T-SRAM und die Transistoren und Kondensatoren bei 1T1C-DRAM. Warum folgt daraus noch keine genaue Chipfläche? (2 P)
3. Ein ausdrücklich angenommener gemeinsamer Port bedient entweder Befehlsholen oder einen Datenzugriff im betrachteten Takt. Beide werden gleichzeitig angefordert. Was ist der Konflikt? Beweist ein gemeinsamer Adressraum allein denselben Konflikt? (2 P)

Musterlösung 1

1. $2^{20} = 1\,048\,576$ Byte, mal acht $= 8\,388\,608$ Bit. Ein MB sind dagegen 10^6 Byte. (1 P)
2. SRAM: $6 \cdot 8\,388\,608 = 50\,331\,648$ Zelltransistoren. DRAM: $8\,388\,608$ Transistoren und ebenso viele Kondensatoren. (1 P) Peripherie fehlt; Bauteilgrößen und Kondensatorgeometrie unterscheiden sich. Die Zählung bestimmt deshalb keine genaue Fläche. (1 P)
3. Beide Anfragen benötigen dieselbe beschränkt verfügbare Ressource; eine muss warten oder später bedient werden. (1 P) Ein gemeinsamer Adressraum legt Portzahl, Caches und physische Wege nicht allein fest. Der Konflikt folgt hier aus dem Ein-Port-Modell. (1 P)

Aufgabe 2: Flashänderung und Ausführungsweg (5 Punkte)

Logisches Flashmodell: gelöscht ist `FF`. Programmieren darf nur eins nach null ändern. Ein Löschen setzt alle Byte eines Modellblocks auf `FF`. Das ist keine Freigabe für beliebige Teilprogrammierung realer Bauteile.

1. Schreiben Sie `A5`, `81` und `AF` binär. Ist `A5` nach `81` ohne Löschen möglich? Und `A5` nach `AF`? (2 P)
2. Im selben Löschblock liegt ein benötigtes Byte `3C`. Beschreiben Sie ein korrektes Update des ersten Bytes von `A5` nach `AF`. (1 P)
3. Sechs **physische Löschzyklen** werden entweder als $6, 0, 0$ oder $2, 2, 2$ auf drei gleichartige Blöcke verteilt. Was verbessert sich, was wird dadurch nicht abgeschafft? (1 P)
4. Bedeutet ein NOR-Flash automatisch, dass daraus Code ausführbar ist? Nennen Sie die nötige Systembedingung für XIP, Execute-in-Place. (1 P)

Musterlösung 2

1. `A5` = `10100101`, `81` = `10000001`, `AF` = `10101111`. `A5` nach `81` setzt nur weitere Einsen auf null: im Modell erlaubt. `A5` nach `AF` verlangt auch null nach eins: vorher löschen. (2 P)
2. Alle weiterhin benötigten Blockinhalte sichern, den ganzen Block löschen, `AF` und das gesicherte `3C` sowie weitere benötigte Inhalte wieder programmieren. Nur das erste Byte wiederherzustellen würde `3C` verlieren. (1 P)
3. Die Summe bleibt sechs, die maximale Belastung sinkt von sechs auf zwei. Endliche Belastbarkeit bleibt; Zuordnung und Updateaufwand sind nötig. Hostwrites sind nicht automatisch physische Löschzyklen. (1 P)
4. Nein. Codeadressen müssen über einen speicherabbildbaren, ausführbaren Leseweg mit passendem Controller und Zugriffsrechten erreichbar sein. Die NOR-Zellverschaltung allein beweist diesen Weg nicht. (1 P)

Aufgabe 3: SRAM-Zustände und Reihenfolge (6 Punkte)

Die klassische 6T-Zelle besitzt zwei kreuzgekoppelte CMOS-Inverter und zwei nMOS-Zugriffstransistoren. CMOS = Complementary Metal-Oxide-Semiconductor. pMOS leitet im qualitativen Modell bei Gate null, nMOS bei Gate eins. Links steuert QB die Transistoren P_Q und N_Q, rechts Q die Transistoren P_QB und N_QB.

1. Füllen Sie die Tabelle für die zwei stabilen Zustände aus. Warum sind null/null und eins/eins keine stabilen digitalen Paare? (2 P)
2. Beschreiben Sie für Q null die Lesefolge mit Vorladung, WL und Leseverstärker. Welche Bitleitung sinkt? Muss Q vollständig unverändert in Volt bleiben? (2 P)
3. Schreiben Sie Q von null nach eins. Welche äußeren Bitleitungswerte werden benötigt? Ordnen Sie Datenbereitstellung, WL-Anstieg, Umschalten, WL-Abfall und Treiberfreigabe. (2 P)

Q	QB	P_Q	N_Q	P_QB	N_QB
0	1				
1	0				

Musterlösung 3

Q	QB	P_Q	N_Q	P_QB	N_QB
0	1	sperrt	leitet	leitet	sperrt
1	0	leitet	sperrt	sperrt	leitet

1. Jeder Inverter muss den Gegenwert seines Eingangs liefern. Gleiche digitale Knotenwerte erfüllen dies nicht gleichzeitig. (2 P)
2. Beide Bitleitungen hoch vorladen, Vorladung ausschalten, WL anheben, Differenz erkennen, WL absenken, danach erneut vorladen. (1 P) BL am Nullknoten sinkt leicht. Q kann einen kleinen analogen Hub besitzen, muss aber bei ausreichender Lesestabilität logisch null bleiben. (1 P)
3. BL eins, BLB null vor WL-Anstieg bereitstellen. Der Treiber zieht QB herunter, Rückkopplung lässt Q steigen. WL erst nach ausreichendem Umschalten absenken, Daten bis danach halten, anschließend Treiber freigeben. (2 P)

Aufgabe 4: DRAM-Ladung und Adressorganisation (6 Punkte)

Idealmodell: $C_{\text{cell}} = 30$ fF, $C_{\text{BL}} = 300$ fF, Plattenspannung null Volt und Bitleitung vorab 0,6 Volt. Die Vorsilbe f in fF bedeutet femto, 10^{-15}. Die Zelle beginnt wahlweise bei 1,2 oder 0 Volt. Ohne Verstärker gilt nach Verbinden:

$$V_{\text{share}} = \frac{C_{\text{cell}} V_{\text{cell}} + C_{\text{BL}} V_{\text{pre}}}{C_{\text{cell}} + C_{\text{BL}}}.$$

1. Berechnen Sie beide gemeinsamen Spannungen und die jeweilige Abweichung vom Vorladepegel. Warum sind Verstärken und Wiederherstellen nötig? (3 P)
2. Ordnen Sie Restore, WL-Absenken und Precharge. Wird beim Lesen der Zellkondensator immer vollständig entladen? (1 P)
3. Adressmodell: vier Banken, je 128 Zeilen, je 64 Spalten, je zwei Byte pro Spaltenelement. Bankbits $[15:14]$, Zeilenbits $[13:7]$, Spaltenbits $[6:1]$, Bytebit $[0]$. Bestimmen Sie Kapazität und Adresse für Bank zwei, Zeile fünf, Spalte neun, Byte eins. (2 P)

Musterlösung 4

1. Hoher Anfangszustand: $(30\cdot 1{,}2 + 300\cdot 0{,}6)/(30+300) = 216/330 \approx 0{,}6545$ V. Niedriger Anfangszustand: $180/330 \approx 0{,}5455$ V. (2 P) Abweichungen etwa $\pm 54{,}5$ mV. Der ursprüngliche Zellpegel wird zum mittleren Pegel hin verändert; der Verstärker erkennt die kleine Richtung und stellt einen deutlichen logischen Zustand wieder her. (1 P)
2. Restore bei noch hoher WL, dann WL absenken, danach BL vorladen. Nein: Die hohe Zelle verliert Ladung, die niedrige erhält im Modell Ladung. Precharge ist nicht Restore. (1 P)
3. $4 \cdot 128 \cdot 64 \cdot 2 = 65\,536$ Byte $= 64$ KiB. (1 P) $2 \cdot 2^{14} + 5 \cdot 2^{7} + 9 \cdot 2 + 1 = 33\,427 =$ `0x8293`. (1 P) Die Bitfeldreihenfolge ist vorgegeben, keine universelle Hardwarezuordnung.

Aufgabe 5: Memory Map und Firmwareanfang (6 Punkte)

Byteadressiertes Lehrmodell: Flash ab `0x00000000`, 256 KiB; SRAM ab `0x10000000`, 64 KiB. Für einen `uint32_t`-Zugriff sind vier Byte und Ausrichtung durch vier nötig.

1. Geben Sie für beide Bereiche das exklusive Ende und das letzte Byte an. (1 P)
2. Prüfen Sie Vier-Byte-Zugriffe ab `0x1000FFFC`, `0x1000FFFF` und `0x10010000` getrennt auf Bereich und Ausrichtung. (1 P)
3. Firmwaremodell ohne Padding und weitere Abschnitte: `.text` 24 KiB, `.rodata` 4 KiB, `.data` 256 Byte, `.bss` 4 KiB und Stack 2 KiB. `.data` hat Initialwerte im Flash und Laufzeitobjekte im RAM; `.bss` enthält keine Nullbytes im Flash-Abbild. Berechnen Sie Bedarf und freien Platz beider Speicher. (2 P)
4. Wer kopiert `.data` und nullsetzt `.bss`? Erzwingt C-`const` allein die Platzierung im Flash? (2 P)

Musterlösung 5

1. Flash: Ende `0x00040000`, letztes Byte `0x0003FFFF`. SRAM: Ende `0x10010000`, letztes Byte `0x1000FFFF`. (1 P)
2. `0x1000FFFC`: vollständig enthalten und ausgerichtet. `0x1000FFFF`: nicht vollständig enthalten und unausgerichtet. `0x10010000`: ausgerichtet, aber bereits außerhalb. (1 P)
3. Flashbedarf $24 \cdot 1024 + 4 \cdot 1024 + 256 = 28\,928$ Byte $= 28{,}25$ KiB; frei 227,75 KiB. (1 P) RAM-Bedarf $256 + 4096 + 2048 = 6400$ Byte $= 6{,}25$ KiB; frei 57,75 KiB. `.data` zählt in beiden Speicherbilanzen. (1 P)
4. Startcode führt Kopieren und Nullsetzen vor der Anwendung aus; der Linker bestimmt die Anordnung und nötigen Adressinformationen. (1 P) `const` beschränkt Änderungen im C-Programm. Abschnittszuordnung, Linkerskript und Plattform bestimmen den Speicherort. (1 P)

Aufgabe 6: FIFO und Speicherwahl (4 Punkte)

FIFO bedeutet First In First Out. Das Modell hat vier nutzbare Arrayplätze, Leseindex r, Schreibindex w und Belegungszahl count. Anfang: r = w = count = null. Bei Erfolg steigt die entsprechende Position modulo vier; bei voll oder leer bleibt ein abgewiesener Vorgang ohne Zustandsänderung. Alle Vorgänge erfolgen in einem Ausführungskontext.

1. Füllen Sie die Zustandsfolge aus. Geben Sie am Ende die Entnahmereihenfolge an. (2 P)
2. Ein weiteres Einlegen von F wird versucht. Ergebnis und Zustand? Warum beweist r gleich w allein keine Leere? (1 P)
3. Firmware soll Abschalten überstehen, häufig veränderte Zwischensummen nicht. Nennen Sie passende Speicherprinzipien und erklären Sie, warum „Code“ und „Daten“ keine Zellbauarten sind. (1 P)

Vorgang	ausgegeben	r	w	count
Anfang	–	0	0	0
A einlegen	–			
B einlegen	–			
C einlegen	–			
entnehmen				
D einlegen	–			
E einlegen	–			

Musterlösung 6

Vorgang	ausgegeben	r	w	count
Anfang	–	0	0	0
A einlegen	–	0	1	1
B einlegen	–	0	2	2
C einlegen	–	0	3	3
entnehmen	A	1	3	2
D einlegen	–	1	0	3
E einlegen	–	1	1	4

1. Entnahmereihenfolge B, C, D, E. (2 P)
2. Voll: F wird abgewiesen; r eins, w eins, count vier bleiben erhalten. Die Anfangspositionen waren ebenfalls gleich, aber count null. Die Belegungszahl unterscheidet beide Fälle. (1 P)
3. Nichtflüchtiger Speicher etwa Flash für das dauerhaft vorhandene Programmbild, SRAM für die häufig geänderten Zwischensummen. Ausführung aus Flash verlangt zusätzlich einen geeigneten Leseweg. Nutzungsart und Zellphysik sind verschiedene Eigenschaften. (1 P)

Optional: logische Modelle mit Host-GCC prüfen

Vom Repository-Hauptordner aus:

```
gcc -std=c11 -O2 -Wall -Wextra -Werror -pedantic \
    labs/src/lab01_memory.c -o /tmp/lab01_memory
/tmp/lab01_memory
```

Erwartete Ausgabe:

```
Flashmodell: A5 -> 81 erlaubt, A5 -> AF braucht Loeschen
Adressmodell: letzter ausgerichteter Wortstart 1000FFFC
Startupmodell: Initialwerte kopiert, BSS auf null
FIFO-Modell: volle und leere Zustaende korrekt
Alle Prueffaelle bestanden.
```

Untersuchen Sie die Fehlerfälle: Ein abgewiesenes Programmieren oder Einlegen ändert den Speicherzustand nicht. Das Programm dereferenziert keine Geräteadressen und modelliert keine analogen Zellströme oder reale Flash-Controller. Es arbeitet ohne Nebenläufigkeit.

Die typische Abschnittszuordnung der C-Folie können Sie unabhängig ansehen:

```
gcc -std=c11 -O0 -fno-pie -c labs/src/lab01_sections.c \
    -o /tmp/lab01_sections.o
readelf -SW /tmp/lab01_sections.o
readelf -sW /tmp/lab01_sections.o
```

Suchen Sie `.text`, `.rodata`, `.data` und `.bss` sowie die vier Symbole. Dies ist ein natives Host-Objekt, keine RISC-V-Firmware und kein Bootversuch. Die konkrete Toolchain kann die Abschnitte anders organisieren.

Hinweise zur optionalen Aufgabe

`flash_program` prüft im logischen Modell vor dem Schreiben, ob alle gewünschten Einsen bereits Einsen sind. Ein Löschen setzt den ganzen Modellblock auf `FF`; das Beispiel stellt anschließend das gesicherte Nachbarbyte `3C` wieder her. Reale Programmiergranularität, Stromausfallfestigkeit und Treiberbefehle sind nicht modelliert.

`word_start_valid` prüft numerische Grenzen, vier Byte Länge und Ausrichtung mit Subtraktion; es erzeugt keine ungültigen C-Zeiger. Das Startupmodell kopiert zwischen gültigen, getrennten Host-Arrays und nullsetzt einen weiteren Bereich. Es richtet keinen Zielprozessor ein.

Der FIFO verwendet einen Belegungszähler und weist ungültige Operationen vor jeder Zustandsänderung ab. Ein r gleich w ist sowohl leer als auch voll möglich. Die Tests umfassen Umbruch, Ablehnung, erneutes Einlegen und komplette Entnahmereihenfolge. Ein anderer Kontext oder Takt bräuchte zusätzliche Synchronisation.

Im unoptimierten Abschnittsbeispiel sind lookup, counter, buffer und read_snapshot in einer typischen GNU-Werkzeugkette den vier genannten Abschnitten zugeordnet. Die Symboltabelle verbindet Namen und Abschnittsnummern. Das endgültige Ladebild und die Laufzeitadressen entstehen erst beim Linken.